

# Daily Scholar Papers Report — 2026-09-01

---

2026-09-01

## Daily Scholar Papers Report — 2026-09-01

---

[Download PDF](#)

**Window covered:** 2026-08-31 → 2026-09-01 (Google Scholar alerts + user-curated self-emails, last 24 h), extended by a 7-day liveness sweep that recovered three alert threads from 25–27 August never previously triaged.

---

### Executive Summary

---

Thirteen papers through screening, and the argument they make together is about **validation** — specifically, about what happens when the thing being validated is generated by a language model and the oracle was designed for code written by a human.

Three papers, arrived independently, attack the same oracle from three sides. An LLM repair agent that patches a memory-corruption bug is normally accepted if the proof-of-concept stops crashing.

**CodeMechanic** names that oracle weak and shows the failure mode: a patch can silence the observed crash by changing behaviour that has nothing to do with the bug. Its response is not a better oracle but a *smaller hypothesis space* — reconstruct the violated memory-safety property from the crash, validate the pointer and its buffer range, insert a local fail-stop guard, and deliberately trade availability for security while a human prepares the real fix. On 101 real-world ARVO bugs that yields **47.6 % more plausible patches than the best baseline using 91 % fewer tokens**, and a manual audit finds **3.4x–4.3x more patches semantically equivalent to developer-written repairs**. A COLM paper on **red-teaming APR agents** comes at the same joint from the security side, asking what a patch that passes its tests has actually established. And **EAHC**, on Verilog generation, dissects the two candidate oracles directly: execution signals are deterministic but limited by testbench coverage, LLM-as-a-Judge is semantically rich but hallucinates on execution-equivalent code. The paper's sharpest empirical point is negative — letting the reasoner see the execution results merely anchors its judgement to the test outcome, collapsing two signals into one. So the two are acquired independently and fused only at the decision stage.

Against that, the most useful result of the window is a paper that discovered its own scaffolding was unnecessary. **Risse and Böhme** set out to beat SZZ — the 2005 bug-introducing-commit heuristic whose authors have just taken the 2026 SIGSOFT Impact Award, and whose successors have moved F1 by only 5–10 points in twenty years. Their SZZ-Agent, a two-stage pipeline of SZZ candidate filtering plus agentic binary search over file history, reaches **0.77 F1 on the Linux dataset against 0.64 for the 2025 state of the art**. Then the ablation labelled, in the paper's own heading, the most surprising finding: raising the binary-search threshold to infinity — that is, **deleting the binary search entirely** — *improves* F1 to 0.82 and cuts cost per fix commit from \$0.67 to \$0.33. The second half of the paper is consequently an argument against its own first half. Hand a stock coding agent the commit history and ordinary developer tools — read, grep, glob — and it scores **0.81 / 0.75 / 0.75 / 0.65** across four

datasets, beating the elaborate pipeline on three of them. The mean agent run uses 9.8 reads and 2.7 greps, and grep patterns are drawn from the fix's removed lines (48.1 %) and commit message (43.6 %) — the agent reinvents SZZ's own move as a search string, then goes further, because unlike SZZ it can handle a fix that only *adds* lines.

Read next to last window's finding — that a model is most useful translating an artefact you already own — this window sharpens it. It is not only that you should not ask the model to invent the artefact. You should also be careful about building machinery around it. Two of the strongest results here were obtained by *removing* structure: EAHC by refusing to let one signal see the other, SZZ-Agent by deleting its own search strategy. The pipeline was the thing standing in the way.

Four further candidates were screened out at Stage-1 as saturated or insufficiently novel; they are not named.

**Outstanding: 3 · Keep: 7 · Borderline High-Priority: 3**

**A note on depth.** Retrieval was constrained this window: the local library held none of these papers, `web_fetch` rate-limited after four documents, and OpenReview served a browser-verification challenge that was not bypassed. One paper was read in full; two were read at full published-abstract depth with verified metadata and licence; the remaining ten are documented at Scholar-alert abstract depth. **Every card states its own evidence base**, and no card claims more than its source supports.

## Highlighted Papers

| Title                                                                               | Authors                                                                      | Venue                                                                                          | Link                                                                                                                                      |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| LLM-Based Agents for Identifying Bug-Introducing Commits                            | N. Risse, M. Böhme                                                           | ASE '26 — 41st IEEE/ACM Int. Conf. on Automated Software Engineering, Munich, Oct 2026, 12 pp. | <a href="https://doi.org/10.1145/3832783.3834337">doi:10.1145/3832783.3834337</a> · <a href="#">author PDF</a> · <a href="#">artifact</a> |
| CodeMechanic: Bug-Property-Guided Program Mitigation                                | H. Zheng, R. Galanopoulou, I. Shumailov, T. Fan, A. Hall, D. Maier, M. Payer | arXiv preprint, cs.SE / cs.CR, 23 Aug 2026                                                     | <a href="https://arxiv.org/abs/2608.22275">arXiv:2608.22275</a>                                                                           |
| Execution-Anchored Hallucination Calibration Reranking for Verilog Code Generation  | G. Yang, X. Hu, X. Chen, T. Y. Zhuo, X. Xia                                  | arXiv preprint, cs.SE / cs.AR, 24 Aug 2026                                                     | <a href="https://arxiv.org/abs/2608.22938">arXiv:2608.22938</a>                                                                           |
| When Correct Patches Become Vulnerable: Red Teaming LLM-Based Program Repair Agents | S. Chen, Y. He, S. Jana, B. Ray                                              | Third Conference on Language Modeling (COLM 2026)                                              | <a href="#">OpenReview WPGbESS31g</a>                                                                                                     |
| BigCodeArena: A Platform for Executable Code Generation Evaluation                  | T. Y. Zhuo, X. Jin, H. Liu, J. Jiang, T. Liu, C. Gong, B. Bishnoi, et al.    | Third Conference on Language Modeling (COLM 2026)                                              | <a href="#">OpenReview TrI9SvEFHK</a>                                                                                                     |
| Learning from the Test: Self-Referential                                            | J. He, J. Shi, Z. Yang, M. Cheng, D. Lo                                      | arXiv preprint, 2026                                                                           | <a href="https://arxiv.org/abs/2608.22284">arXiv:2608.22284</a>                                                                           |

| Title                                                                                        | Authors                                                      | Venue                                                    | Link                                                              |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|
| Differential Testing for Deep RL Agents                                                      |                                                              |                                                          |                                                                   |
| Towards LLM-Enhanced Android Taint Analysis                                                  | N. Miazza, M. Alecci, J. Samhi, J. Klein, E. Losiouk         | arXiv preprint, 2026                                     | <a href="https://arxiv.org/abs/2608.24269">arXiv:2608.24269</a>   |
| Effective Fuzzing within CI/CD Pipelines                                                     | A. Sharma, C. Cadar, J. Metzman                              | ACM Trans. on Software Engineering and Methodology, 2026 | <a href="https://doi.org/10.1145/3841630">doi:10.1145/3841630</a> |
| FlowScout: From Execution Feedback to Reliable Tool-Using Agent Workflows                    | S. Hao, Y. Lu, B. Chen, X. Peng                              | arXiv preprint, 2026                                     | <a href="https://arxiv.org/abs/2608.10039">arXiv:2608.10039</a>   |
| DOCSCHISEL: Adaptive Tool Documentation Optimization Framework for LLM Agents                | Y. Lu, K. Zhang, B. Chen, X. Peng                            | arXiv preprint, 2026                                     | <a href="https://arxiv.org/abs/2608.10037">arXiv:2608.10037</a>   |
| SoK: ARCUS: On the Efficiency and Efficacy of Hardware Fuzzing                               | A. K. Murali, R. Saravanan, A. Venkat                        | arXiv preprint, 2026                                     | <a href="https://arxiv.org/abs/2608.23933">arXiv:2608.23933</a>   |
| Towards Robust Android Malware Detection under Concept Drift and Code Obfuscation            | H. Yu, T. Wang, L. K. Shar, J. Liu, D. Lo                    | IEEE Trans. on Software Engineering, 2026                | <a href="https://doi.org/10.1109/11660368">IEEE 11660368</a>      |
| ASFuzz: Detecting Linux Kernel Asynchronous Concurrency Bugs via Asynchronous Shadow Fuzzing | J. Zhao, Y. Geng, J. Wang, Y. Chen, H. Chen, Y. Wang, et al. | IEEE Transactions, 2026                                  | <a href="https://doi.org/10.1109/11660760">IEEE 11660760</a>      |

## Papers

**1.1 · MINING SOFTWARE REPOSITORIES** · Deleting the binary search improves F1 from 0.81 to 0.82 and halves the cost

### LLM-Based Agents for Identifying Bug-Introducing Commits

**Authors:** Niklas Risse, Marcel Böhme (MPI-SP and CISPA, Germany)

**Venue:** ASE '26 — Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering, Munich, Germany, 12–16 October 2026, 12 pages. Received 2026-03-26; accepted 2026-06-18.

**Links.** [ACM DOI 10.1145/3832783.3834337](https://doi.org/10.1145/3832783.3834337) · [author-hosted PDF](#) · [artifact](#), [Zenodo](#). **License: CC BY 4.0** — the front matter states the work is licensed under a Creative Commons Attribution 4.0 International License, copyright held by the authors. **Full text read (12 pp., all tables and appendices).**

## The problem, stated as a stagnation

Identifying which commit introduced a bug determines which released versions are affected — a question with direct security consequences. SZZ has answered it since 2005, and its authors have just received the 2026 ACM SIGSOFT Impact Award for it. The paper’s framing is that this is precisely the problem: on the developer-annotated Linux dataset of Lyu et al., the original SZZ scores **0.54 F1**, and the 2025 state of the art (LLM4SZZ, ISSTA ’25) scores **0.64** — ten points in twenty years. Across all five datasets used here the cumulative gain over original SZZ stays within **5–10 percentage points**.

The diagnosis is structural, not parametric. SZZ assumes the lines *deleted* by a fix are the lines that introduced the bug. The paper’s counterexample is clean: a commit adds a function that fails to validate its inputs; the fix *adds* a validation check rather than touching the original function. The deleted lines are then unrelated to the real culprit — and in the limiting case, a fix with zero deletions gives SZZ nothing to trace at all. Refactoring noise compounds it: prior work (RA-SZZ) attributes **6.5 %** of incorrect bug-introducing flags to behaviour-preserving changes.

The task is defined in prose, not in a numbered environment: given a fix commit  $c_{\text{fix}}$ , identify the set of bug-introducing commits  $B \subseteq H_B \subseteq H$  where  $H$  is the prior history;  $|B|/|B|$  is usually one but need not be.

## Two tools, and the second one refutes the first

**SZZ-Agent** is a two-stage pipeline. Stage 1 runs standard SZZ to produce candidates, then hands the agent the fix message, the fix diff, and each candidate’s message and diff with five lines of context, asking for the earliest responsible commit or an explicit abstention. Stage 2 fires when Stage 1 abstains or SZZ produced nothing: binary search over the timestamp-ordered history of every file the fix touched, the agent judging at each step whether the bug is already present, switching to direct selection once the window narrows past a tunable threshold.

**Simple-SZZ-Agent** is what the ablations produced. No SZZ filtering, no binary search, no stages: collect the commit histories of all files the fix modified into one candidate set, hand it to the agent together with the fix message and diff, and give it ordinary developer tools — file reading, grep, glob.

Leakage controls are unusually thorough and worth copying: ground-truth commit hashes replaced with `COMMIT_HASH` throughout diffs and messages, `Fixes:` lines removed entirely, the agent given a working directory of loose files rather than a real git repository, and internet access plus the `WebSearch/` `WebFetch` tools disabled.

Evaluation metric, **Equation (1) of the paper** — the only numbered equation it contains:

$$\text{Precision}(c_{\text{fix}}) = \frac{|B \cap \hat{B}|}{|\hat{B}|}, \text{Recall}(c_{\text{fix}}) = \frac{|B \cap \hat{B}|}{|B|} \quad \text{Precision}(c_{\text{fix}}) = \frac{|B \cap \hat{B}|}{|\hat{B}|}, \quad \text{Recall}(c_{\text{fix}}) = \frac{|B \cap \hat{B}|}{|B|}$$

macro-averaged across fix commits, with F1 the harmonic mean of the averaged values. *(The paper contains no numbered definitions, algorithms, theorems or lemmas — verified against the full text.)*

## What the numbers say

**RQ1, F1 against eight baselines.** SZZ-Agent scores **0.77 / 0.75 / 0.67 / 0.60** on DS\_LINUX, DS\_GITHUB-c, DS\_GITHUB-j and DS\_APACHE, against LLM4SZZ’s 0.64 / 0.63 / 0.55 / 0.51 — gains of

**13, 12, 12 and 9 points.** Significant at  $p < 0.01$  with large effect sizes, with one honestly-reported exception where the effect size is large but  $p = 0.013$ .

**Leakage ruled out empirically.** A new dataset, DS\_LINUX-26, built from fixes published between 1 September 2025 and 31 January 2026 — after the backbone model’s stated August 2025 cutoff. SZZ-Agent scores **0.81** there, *higher* than on the older dataset. Baselines move by a point or two.

**The backbone is not the story.** Given the same claude-opus-4-5, LLM4SZZ reaches 0.65 and SZZ-Agent 0.77. The architecture, not the model, carries the 12 points.

**Context ablation.** Neither message nor diff: **0.48**, below plain SZZ. Diff only: 0.73. Message only: 0.77. Both: 0.81. The commit message alone outperforms the diff alone — a result worth sitting with, given that SZZ is a diff-based method by construction.

**The finding the paper itself labels most surprising.** Sweeping the binary-search threshold:

| Threshold            | Precision | Recall | F1          | Cost / fix commit |
|----------------------|-----------|--------|-------------|-------------------|
| 8                    | 0.76      | 0.75   | 0.76        | \$0.68            |
| 32                   | 0.81      | 0.80   | 0.81        | \$0.67            |
| 128                  | 0.80      | 0.79   | 0.80        | \$0.39            |
| 512                  | 0.81      | 0.80   | 0.81        | \$0.35            |
| ∞ (no binary search) | 0.82      | 0.81   | <b>0.82</b> | <b>\$0.33</b>     |

The paper’s own contribution is dominated by its removal — better F1 at roughly half the cost, the abstract’s **43 % token-cost reduction**.

**Simple-SZZ-Agent** then beats SZZ-Agent on three of four datasets: **0.81 vs 0.77**, 0.75 vs 0.75, **0.75 vs 0.67**, **0.65 vs 0.60**.

**How the agent actually works, measured rather than asserted.** Mean tool calls per run: Read 9.8, Grep 2.7, Write 1.0, Glob 0.8. Grep patterns originate from the fix’s removed lines (**48.1 %**) and its commit message (**43.6 %**). Across 243 grep calls the median pattern is 21 characters and **63.8 % are plain strings, not regexes**. The authors note the removed-lines case is SZZ’s own move performed with grep instead of `git blame` — and the worked pure-addition example (3 insertions, 0 deletions) is one SZZ cannot attempt. Scaling is close to flat: F1 stays above 0.80 up to 1,000 candidates, falling to **0.72** beyond that, with cost essentially uncorrelated with candidate count ( $r = -0.03$ ).

**Error analysis, 100 commits.** 87 correct. Of 13 failures: 4 where the true commit was not in the candidate set, 9 selection errors — 3 faulty analysis, 2 near misses within three positions, **3 where the authors judge the ground-truth label itself questionable**, 1 undetermined.

**Frameworks and models.** Best configuration is OpenHands with claude-sonnet-4-5 at **0.86 F1**; the cheapest respectable one is OpenHands with minimax-m2.5 at 0.79 F1 for **\$0.04** per fix commit. The authors immediately caveat their own best number: the OpenHands runs consume three to four times more tokens, so the advantage may be spend rather than architecture.

## Reading it critically

The appendix is where the honest reader should go. Both agents only ever search files the fix touched, so when the bug-introducing commit lives in an untouched file they are **N/A by construction** — 5.5 % to 13.2 % of cases depending on dataset, counted as failures in the headline tables. On the function-level-overlap split, Simple-SZZ-Agent drops from 0.89 to 0.75 on Linux and from 0.86 to 0.52 on the Java set. The performance is real, but it is concentrated where the fix and the bug touch the same code, and the authors say so.

Two internal inconsistencies survive into the camera-ready: the binary-search threshold is given as 33 in the text and figure but swept as 32 in the table, and the Java dataset is described as 75 samples while the appendix tables sum to 73. Neither affects the argument.

The threats section is unusually self-implicating. The authors apply the first author's own prior critique of vulnerability-detection benchmarks to this paper, concede the agents may be exploiting superficial cues, and then make the one move that actually answers it: this benchmark is not a proxy for some other capability, it *is* the deployment task. They decline to claim the agents understand bugs.

On the strategic point, their own framing is hard to improve on: *"To break the trend of small incremental improvements, entirely new approaches are needed."*

**1.2 · AUTOMATED PROGRAM REPAIR ·** Stop trying to repair the bug: insert a fail-stop guard, 47.6 percent more plausible patches at 91 percent fewer tokens

### **CodeMechanic: Bug-Property-Guided Program Mitigation**

**Authors:** Han Zheng, Rafaila Galanopoulou, Ilia Shumailov, Tianqi Fan, Aiden Hall, Dominik Maier, Mathias Payer

**Venue:** arXiv preprint arXiv:2608.22275, cs.SE / cs.CR, submitted 23 August 2026.

**Links.** [arXiv:2608.22275](https://arxiv.org/abs/2608.22275) · [PDF](#) · [HTML](#). **License:** **arXiv non-exclusive distribution 1.0** — not a Creative Commons class licence, so no mirrored PDF and no figure reproduction here. **Read at full published-abstract depth**, with authorship, subject class and licence verified against the arXiv record; section-level detail not retrieved.

## The gap being targeted

The framing is operational rather than academic. Automated testing now finds memory-corruption bugs faster than developers can trace and repair them, which leaves a window in which a known, exploitable defect sits in shipped code. End-to-end LLM repair agents can shorten that window — but they generate open-ended code changes and typically validate them by replaying the proof-of-concept. The paper's charge against that oracle is specific: it accepts patches that silence the observed crash by altering unrelated behaviour. A patch that passes has demonstrated that this PoC no longer crashes, and nothing else. Deploying it is a gamble.

## The move: constrain the output, not the oracle

Rather than strengthen validation, CodeMechanic shrinks what the model is allowed to produce. It does not ask for a permanent repair at all. From the crash it reconstructs the violated memory-safety property,

validates the dereferenced pointer and its buffer range, and inserts a **local fail-stop guard** immediately before the dangerous access. When the boundary check fails, execution terminates.

The trade-off is stated rather than hidden: **availability is spent to buy security**. A potential remote code execution becomes a controlled termination, and the window stays covered while developers do the root-cause work and write the real fix. That reframing is the contribution — mitigation as a first-class, temporary, deliberately incomplete artefact, distinct from repair.

Three components keep LLM error bounded: a two-dimensional static-and-dynamic context extractor, in-prompt debugging knowledge, and stepwise validation.

## What the numbers say

On **101 real-world ARVO bugs**, CodeMechanic's *first attempt* produces **47.6 % more plausible patches** — patches passing PoC-replay validation — than the strongest baseline, while consuming **91 % fewer tokens**. The token figure is the one to note: a narrower task specification is dramatically cheaper than open-ended generation, and the paper gets both the accuracy and the cost on the same side of the ledger.

A manual audit goes past the plausibility oracle the paper spent its introduction criticising: CodeMechanic yields **3.4x to 4.3x more patches semantically equivalent to developer-written repairs**. Reporting that separately from the plausibility rate is the right call, since plausibility is exactly the metric under suspicion.

## Why it is worth your time

The bug-property reconstruction step is the reusable part, independent of the LLM framing: it converts a crash into an explicit safety property, which is a well-defined artefact that other tooling can consume. The narrower observation — that constraining the hypothesis space beats strengthening the oracle when the oracle is fundamentally weak — generalises well past memory safety.

Two things the abstract does not settle, and which the full text would need to answer before adoption: the runtime and code-size cost of the inserted guards, and the false-positive rate of the boundary checks on benign inputs. A fail-stop guard that fires in production on legitimate traffic converts a latent vulnerability into an immediate outage.

**1.3 · LLM CODE GENERATION ·** Show the judge the test results and it stops judging: two signals fused only at the decision stage

## Execution-Anchored Hallucination Calibration Reranking for Verilog Code Generation

**Authors:** Guang Yang, Xing Hu, Xiang Chen, Terry Yue Zhuo, Xin Xia

**Venue:** arXiv preprint arXiv:2608.22938, cs.SE / cs.AR, submitted 24 August 2026.

**Links.** [arXiv:2608.22938](https://arxiv.org/abs/2608.22938) · [PDF](#) · [HTML](#). **License:** CC BY 4.0 per the arXiv licence record. **Read at full published-abstract depth;** authorship, subject class and licence verified against the arXiv record. Section-level detail and evaluation tables not retrieved — the numeric claims below are the ones the abstract states.



## The setting

LLMs generate code well and Verilog badly, because hardware description languages are low-resource. Sampling many candidates raises the chance that a correct one exists; it does nothing to tell you which. Reranking is therefore the whole game, and the paper's contribution is an empirical study of why the two obvious rerankers both fail, followed by an architecture derived from that failure.

## The empirical finding, which is the real contribution

Across **nine models and two benchmarks**, two limitations are isolated:

**Execution-based reranking** — accept or reject on testbench pass/fail — transfers poorly across domains, because the generated testbenches are themselves low quality. The signal is deterministic but its coverage is bounded by an artefact that is as hard to synthesise as the code under test.

**LLM-as-a-Judge** suffers reasoning hallucination: it returns inconsistent verdicts on code that is *execution-equivalent*. Two implementations that behave identically receive different scores, which is disqualifying for a ranker.

The paper's framing of what to do about it is the part worth stealing: these are **two signal types with orthogonal errors** — execution deterministic but coverage-limited, reasoning semantically rich but hallucination-prone. Orthogonal errors are the textbook precondition for ensembling, so combine them.

## The negative result inside the positive one

The obvious combination is to let the reasoning model observe the execution results. The authors tried it and report that it fails, for an interesting reason: the reasoner's judgements simply become **anchored on the test outcomes**. Conditioning one signal on the other does not fuse them, it collapses them — you have re-implemented execution-based reranking with extra steps and lost the independent semantic channel entirely.

The design follows directly: acquire the two signals **independently**, and fuse only at the decision stage. This is late fusion justified by a measured failure of early fusion, not by convention.

**EAHC** implements it as a dual channel: **EAHC-R**, a 4B reasoning discriminator, and **EAHC-T**, a testbench generator using retrieval-augmented generation for execution verification. The 4B discriminator is notable — the semantic channel does not need a frontier model once it is not also being asked to simulate execution.

## Why it is worth your time

The transferable lesson is about pipeline architecture rather than Verilog. Any time two validators are available and one is cheap-and-brittle while the other is rich-and-unreliable, the instinct is to let the smart one look at the cheap one's output. This paper is evidence that the instinct is wrong, and that independence is a property worth protecting even at the cost of a less informed judge.

Being a preprint at abstract depth, the ablation isolating the fusion-stage choice — and the size of the gain over each channel alone — is exactly what a full read should check first.

**1.4 · PROGRAM REPAIR SECURITY** · A patch that closes the GitHub issue and opens a vulnerability still counts as correct



## When Correct Patches Become Vulnerable: Red Teaming LLM-Based Program Repair Agents

**Authors:** Sicheng Chen, Yifan He, Suman Jana, Baishakhi Ray

**Venue:** Third Conference on Language Modeling (COLM 2026).

**Links.** [OpenReview forum WPGbESS31g](#) · [PDF](#). **Evidence base: Scholar-alert abstract only.**

OpenReview served a browser-verification challenge which was not bypassed, so the paper body, evaluation setup and numbers were not retrieved. Licence not determined.

LLM agents are being handed GitHub issues and asked to produce patches, and the field's success metric is whether the reported problem goes away. This paper asks the adversarial question that metric cannot see: whether a patch that resolves the issue can simultaneously introduce a vulnerability. The title is the thesis — correctness with respect to the bug report is not correctness with respect to security, and an APR agent optimising the former has no pressure to preserve the latter.

Read alongside CodeMechanic (1.2), which independently argues that PoC-replay validation accepts patches that change unrelated behaviour, the two form a pincer on the same weakness from opposite directions: one shows the oracle admits *harmless* wrong patches, the other that it admits *harmful* ones. Ray's group has a track record on the security-of-generated-code question, and Jana's presence signals the threat modelling is likely to be serious rather than illustrative.

Worth a full read when the PDF becomes reachable, specifically for how they operationalise the red team: an attack that requires crafting an adversarial issue is a much weaker result than one showing that ordinary issues already induce vulnerable patches at a measurable rate. That distinction is the whole paper, and the abstract does not settle it.

**1.5 · EVALUATION INFRASTRUCTURE** · Crowdsourced code preference is unreliable when nobody runs the code

## BigCodeArena: A Platform for Executable Code Generation Evaluation

**Authors:** Terry Yue Zhuo, Xiaolong Jin, Hange Liu, Juyong Jiang, Tianyang Liu, Chen Gong, Bhupesh Bishnoi, et al.

**Venue:** Third Conference on Language Modeling (COLM 2026).

**Links.** [OpenReview forum TrI9SvEFHK](#) · [PDF](#). **Evidence base: Scholar-alert abstract only**

(OpenReview not retrievable; see 1.4). Licence not determined. Surfaced by two independent followed-author alerts.

Chatbot Arena made human preference the currency of model evaluation, and it works because a human can read two paragraphs and pick one. The paper's observation is that this breaks for code: outputs are long, and judging them requires reasoning about *execution behaviour* that the annotator cannot perform by reading. Preference data collected without running the program measures something — style, plausibility, familiarity — that is not correctness.

BigCodeArena's answer is to put execution inside the arena, so the human judges observed behaviour rather than source text. As infrastructure this is the more consequential kind of contribution: a platform that changes what the community's preference data means, rather than a model that scores better on the old data.

The direct link to this window's theme is that it is the same oracle problem one level up. EAHC (1.3) found that an LLM judge hallucinates on execution-equivalent code; this paper argues human judges have a version of the same defect, and both conclude that execution has to be a first-class signal rather than something the judge is trusted to simulate mentally.

What a full read needs to establish: the volume and licence of the collected interaction data, whether execution is sandboxed per-language or per-task, and whether human agreement actually improves once execution is visible — that last one is the empirical claim the platform stands or falls on.

**1.6 · TESTING ML SYSTEMS ·** Differential testing without a reference implementation: the agent is its own oracle

### **Learning from the Test: Self-Referential Differential Testing for Deep RL Agents**

**Authors:** Jiahao He, Jiawei Shi, Zhou Yang, Mingyue Cheng, David Lo

**Venue:** arXiv preprint arXiv:2608.22284, 2026.

**Links.** [arXiv:2608.22284](https://arxiv.org/abs/2608.22284) · [PDF](#). **Evidence base:** Scholar-alert abstract only — `web_fetch` was rate-limited before this record could be retrieved. Licence not determined.

Deep RL systems are being deployed into real decision-making, and testing them is genuinely hard for a reason that is structural rather than incidental: differential testing needs two implementations that should agree, and for a trained policy there is no reference implementation to compare against. Metamorphic testing needs relations someone must state. Coverage means little when the artefact is a weight matrix.

The proposed way out is in the title — **self-referential** differential testing, deriving the comparison from the agent itself rather than from an external reference. If that works it is a neat inversion, because the absence of a second implementation is exactly what made the technique inapplicable.

The design question a full read must answer is what plays the role of the second implementation, and whether the disagreements it surfaces are genuine faults or merely the ordinary stochasticity of a policy. A differential tester whose alarms are dominated by seed variance is not useful, and for RL that is the default outcome unless the construction is careful.

Flagged for the methodology rather than the domain: a working recipe for differential testing without a reference oracle would transfer well beyond RL, to any learned component embedded in a larger system.

**1.7 · STATIC ANALYSIS ·** Keep FlowDroid, use the model for the modelling gaps rather than the analysis

### **Towards LLM-Enhanced Android Taint Analysis**

**Authors:** Nicolò Miazzo, Marco Alecci, Jordan Samhi, Jacques Klein, Eleonora Losiouk

**Venue:** arXiv preprint arXiv:2608.24269, 2026.

**Links.** [arXiv:2608.24269](https://arxiv.org/abs/2608.24269) · [PDF](#). **Evidence base:** Scholar-alert abstract only — retrieval blocked by rate limiting. Licence not determined.

Taint analysis for Android has a well-understood ceiling, and the abstract names it correctly: the obstacle is not the dataflow algorithm but the difficulty of accurately *modelling the Android framework* — lifecycle callbacks, ICC, reflection, native boundaries, and the sheer surface area of the SDK. FlowDroid's

imprecision is largely inherited from an incomplete model of an enormous platform, not from the analysis engine.

The word doing the work in the title is **enhanced**. This is positioned as augmentation of a mature static analyser rather than replacement of it by a model, which is the correct architecture and, encouragingly, is becoming the default in this line of work. The interesting engineering question is where the seam is placed: using an LLM to synthesise or complete framework models is a promising and checkable use, since a proposed model can be validated against runtime behaviour; using one to decide whether a flow is a real leak is not, since there is nothing to check it against.

The Luxembourg group (Samhi, Klein) has consistently produced reusable Android analysis artefacts, which raises the prior that the framework-modelling contribution is usable independently of the LLM framing. That detachability is the reason to read it — the model-completion component may be worth more than the end-to-end result.

Relevant to a standing interest here: static-analysis papers have been positively rated in this digest before, and this is the window's only static-analysis entry.

**1.8 · FUZZING ·** Directed fuzzing has a natural home: the CI pipeline already knows what changed

### **Effective Fuzzing within CI/CD Pipelines**

**Authors:** Arindam Sharma, Cristian Cadar, Jonathan Metzman

**Venue:** ACM Transactions on Software Engineering and Methodology, 2026.

**Links.** [doi:10.1145/3841630](https://doi.org/10.1145/3841630) · [ACM PDF](#). **Evidence base:** Scholar-alert abstract only — not deep-read this window. Licence not determined.

The argument is clean and the author list is the reason to take it seriously: Cadar on the analysis side, Metzman from the OSS-Fuzz side, which means the deployment constraints are likely to be real ones. Continuous fuzzing normally runs as a background process against a whole target; the observation here is that CI already tells you which code changed, and directed greybox fuzzers such as AFLGo already know how to prioritise inputs by estimated distance to a target. The fit is natural and, notably, under-exploited in practice.

The hard part is implied rather than solved by the framing: CI budgets are minutes, not machine-days, and distance computation in AFLGo-style fuzzers has a non-trivial instrumentation cost that must be amortised against a target that changes on every commit. A TOSEM-length treatment has room to address exactly that, and it is what to read for.

Practically relevant if you run fuzzing in CI at all: this is the paper to check before building the integration yourself.

**1.9 · AGENT ENGINEERING ·** Build the workflow from execution feedback instead of writing it by hand

### **FlowScout: From Execution Feedback to Reliable Tool-Using Agent Workflows**

**Authors:** Shuang Hao, Yifan Lu, Bihuan Chen, Xin Peng

**Venue:** arXiv preprint arXiv:2608.10039, 2026.

**Links.** [arXiv:2608.10039](https://arxiv.org/abs/2608.10039) · [PDF](#). **Evidence base: Scholar-alert abstract only.** Licence not determined.  
*Recovered by the 7-day liveness sweep — this thread arrived 27 August and had never been triaged by any run.*

Agentic workflows — LLMs, tools and control logic arranged into an explicit execution structure — have become the standard abstraction for building reliable automation, and the abstract's complaint is that *constructing* a good one is still manual. FlowScout's proposal is to derive the structure from execution feedback rather than from a designer's guess.

This is the same shape as the window's central finding. Risse and Böhme (1.1) discovered by ablation that their hand-designed two-stage structure was worse than none at all; FlowScout starts from the premise that hand-designed structure is the problem and tries to learn it from observed behaviour. Both are reacting to the fact that agent scaffolding is currently written on intuition and rarely ablated.

What to check on a full read: whether the learned workflows are actually more reliable than a single-agent-with-tools baseline. That baseline is the one 1.1 shows to be surprisingly strong, and it is the comparison this literature most often omits.

**1.10 · AGENT ENGINEERING** · Tool documentation is a grounding artefact, so optimise it rather than the agent

### **DOCSCHISEL: Adaptive Tool Documentation Optimization Framework for LLM Agents**

**Authors:** Yifan Lu, Kaiyuan Zhang, Bihuan Chen, Xin Peng

**Venue:** arXiv preprint arXiv:2608.10037, 2026.

**Links.** [arXiv:2608.10037](https://arxiv.org/abs/2608.10037) · [PDF](#). **Evidence base: Scholar-alert abstract only.** Licence not determined.  
*Recovered by the 7-day liveness sweep, same thread as 1.9.*

Companion piece to FlowScout from the same lab, and the pairing is the interesting part. Where 1.9 optimises the control structure, this optimises the *grounding resource*: tool documentation. The abstract's framing is that existing work concentrates on improving the agent's tool-use capability and treats the documentation as fixed, when the documentation is a critical part of what the agent is reasoning from.

That is the same insight this digest recorded on 2026-08-30 in a different domain — the ISSTA property-based testing work found its entire gain in annotating widgets before generating anything, with an 18-point accuracy swing attributable to grounding rather than generation. Two labs, two domains, one conclusion: the leverage is in the artefact the model reads, not the model or the prompt.

The evaluation question is whether the optimised documentation transfers across models and agent frameworks, or whether it is being fitted to one model's quirks. Documentation tuned into a per-model artefact is much less useful than documentation that is simply better.

**1.11 · HARDWARE FUZZING** · One taxonomy across ISA, microarchitecture and RTL

### **SoK: ARCUS: On the Efficiency and Efficacy of Hardware Fuzzing**

**Authors:** Aravind Krishnan Murali, Rahul Saravanan, Ashish Venkat

**Venue:** arXiv preprint arXiv:2608.23933, 2026.

**Links.** [arXiv:2608.23933](https://arxiv.org/abs/2608.23933) · [PDF](#). **Evidence base: Scholar-alert abstract only.** Licence not determined.

Systematisation of knowledge covering hardware fuzzing at three abstraction layers — instruction set architecture, microarchitecture, and register-transfer level. Outside the software-engineering core, which is why it sits here rather than higher, but SoKs earn their place through the taxonomy and the common evaluation framing rather than through new results, and a cross-layer comparison is genuinely useful: coverage metrics that mean one thing at RTL mean something quite different at the ISA level, and that mismatch is a recurring source of incomparable numbers in this literature.

Worth skimming if you work on coverage-guided fuzzing generally — the efficiency/efficacy split in the title suggests the authors separate “found more bugs” from “explored more state”, a distinction software fuzzing papers routinely blur.

**1.12 · ANDROID SECURITY · Two robustness problems at once: model aging and deliberate obfuscation**

### **Towards Robust Android Malware Detection under Concept Drift and Code Obfuscation**

**Authors:** Haodong Yu, Tian Wang, Lwin Khin Shar, Jiaqi Liu, David Lo

**Venue:** IEEE Transactions on Software Engineering, 2026.

**Links.** [IEEE Xplore 11660368](https://ieeexplore.ieee.org/abstract/document/11660368) · [Scholar lookup](#). **Evidence base: Scholar-alert abstract only.** Licence not determined. *Recovered by the 7-day liveness sweep — thread arrived 25 August after that day's run and had never been triaged.*

Learning-based malware detection is a crowded area and most entries in it are screened out of this digest. This one is retained on two independent signals: TSE, and a problem statement about *robustness* rather than about another detector. The two challenges named — concept drift causing model aging, and code obfuscation as an adversarial input transformation — are the two reasons deployed detectors decay, and they are usually studied separately.

Treating them jointly is the potentially novel part, because the natural defences pull against each other: adapting continuously to drift makes a model easier to poison, while hardening against obfuscation tends to mean committing to features that drift. A paper that handles both without one undoing the other would be worth the read; one that stacks two independent mitigations would not. The abstract does not distinguish these cases.

**1.13 · KERNEL FUZZING · Delayed triggering is why async kernel concurrency bugs survive conventional fuzzers**

### **ASFuzz: Detecting Linux Kernel Asynchronous Concurrency Bugs via Asynchronous Shadow Fuzzing**

**Authors:** Jian Zhao, Yifan Geng, Jian Wang, Yi Chen, Hao Chen, Yu Wang, et al.

**Venue:** IEEE Transactions, 2026.

**Links.** [IEEE Xplore 11660760](https://ieeexplore.ieee.org/abstract/document/11660760) · [Scholar lookup](#). **Evidence base: Scholar-alert abstract only.** Licence not determined. *Recovered by the 7-day liveness sweep.*

The technical observation is sharp enough to be worth recording even without a full read: concurrency bugs arising from *asynchronous-to-synchronous* interactions are hard to fuzz because the asynchronous task fires on a delay, so the crash is separated in time from the input that caused it. Conventional kernel

fuzzers attribute crashes to the syscall sequence that immediately preceded them, and that attribution is simply wrong for this bug class — which means the bugs are not merely rare in the search, they are systematically mis-credited when found.

“Asynchronous shadow fuzzing” suggests running a shadow execution to make the delayed interaction observable. If the mechanism generalises past the kernel it applies to any event-driven system with deferred callbacks, which is most of them.

Borderline only because it was not deep-read and the venue string from the alert is incomplete. Promote if kernel concurrency is on your critical path.

---

## Cross-Paper Synthesis

---

### The oracle is the bottleneck, and three groups said so independently

The papers in this window were not written together, but four of them are arguing about the same thing: what it means for machine-generated code to be *validated*, and whether the check we are using establishes what we think it does.

The chain is worth laying out because each link fails differently.

**Pass-the-PoC is too weak.** CodeMechanic (1.2) shows the failure directly: a repair agent can make the crash go away by changing behaviour unrelated to the bug, and PoC replay will sign it off. The response is notable for what it *isn't* — no attempt to build a stronger validator. Instead the hypothesis space is narrowed until a weak oracle suffices: reconstruct the violated safety property, emit a fail-stop guard, and accept a controlled termination in place of a repair. The token cost falls by 91 % because a constrained task is a cheaper task, and the audit finding of 3.4×–4.3× more developer-equivalent patches is what tells you the constraint did not merely trade one kind of wrongness for another.

**Pass-the-tests is too weak in the other direction.** The COLM red-teaming paper (1.4) takes the same oracle and shows it admits patches that are actively harmful rather than merely irrelevant. Together with 1.2 this brackets the problem: the oracle used across the APR literature is permissive in both the benign and the adversarial direction, and neither failure is visible in the metric the field reports.

**So use two oracles — but do not let them talk.** EAHC (1.3) is the constructive answer and contains the window's most transferable negative result. Execution signals and reasoning signals have orthogonal errors, which is the standard justification for ensembling them. The authors then tried the obvious implementation — show the reasoning model the execution results — and found it collapses the ensemble, because the reasoner anchors on the test outcome and stops contributing an independent view. Late fusion here is not a design convention; it is a measured consequence of early fusion destroying the second signal.

**And the human oracle has the same defect.** BigCodeArena (1.5) observes that arena-style preference collection assumes an annotator who can judge by reading, which fails for code because correctness is a property of execution. Putting execution in front of the human is the same move EAHC makes for the model judge. Two papers, two kinds of judge, one conclusion: do not ask a judge to simulate execution in its head.

## The scaffolding recession

The second thread cuts against a strong prior in the current literature, which is that agentic systems get better as you add structure.

Risse and Böhme (1.1) provide the cleanest evidence to the contrary, and it is unusually credible because it undermines their own contribution. Their engineered pipeline — SZZ candidate filtering, then agentic binary search over file history — reaches 0.77 F1, comfortably past the 0.64 state of the art. The threshold sweep then shows that setting the binary-search bound to infinity, which is to say removing the search, gives 0.82 F1 at half the cost. Simple-SZZ-Agent, which is a stock coding agent handed the commit history and read/grep/glob, beats the pipeline on three of four datasets.

The mechanism is legible in their instrumentation rather than left to speculation. Roughly ten reads and three greps per run; grep patterns drawn from the fix's removed lines 48.1 % of the time and its commit message 43.6 % of the time; nearly two-thirds of patterns plain strings rather than regexes. The agent rediscovers SZZ's core heuristic — trace the deleted lines — and implements it with grep instead of `git blame`. Then it does the thing SZZ structurally cannot, which is handle a fix consisting only of additions. The scaffolding was encoding a strategy the agent would have found anyway, while constraining it from finding the others.

FlowScout (1.9) and DOCSCHISEL (1.10) approach the same suspicion from the other end: if hand-written scaffolding is unreliable, derive the workflow from execution feedback, and invest in the documentation the agent is grounding on. Which lands on the conclusion this digest recorded on 2026-08-30 from an entirely different set of papers — that the leverage sits in the artefact the model reads rather than in the model or the orchestration.

The sharpened version, after this window: **before reporting that your scaffold works, ablate it to nothing**. The strongest paper here is strong precisely because its authors ran that ablation and published the result that cost them their headline contribution. Two of 1.1's four datasets would have looked better if they had not.

## A methodological note that is not about LLMs

1.1's leakage handling deserves separate mention as reusable practice: ground-truth hashes replaced throughout, `Fixes:` tags stripped, the repository flattened into loose files so the agent cannot run `git log`, network access disabled, and — the part most papers skip — a fresh dataset built entirely from fixes published after the model's training cutoff, on which performance went *up* rather than down. That last step converts “we believe there is no leakage” into a measurement. Any evaluation of an agent against historical repository data should be held to it.

## Writing & Rationale Insights

---

**Piggyback on the news, then use it as the problem statement.** 1.1 opens by noting that SZZ's authors have just won the 2026 SIGSOFT Impact Award — and turns the celebration into the indictment: twenty years, more than 1,300 citations, ten points of F1. The award is what makes the stagnation quantifiable and the paper timely at once. The authors state the consequence plainly: *“To break the trend of small incremental improvements, entirely new approaches are needed.”*

**Publish the ablation that kills your contribution, and label it.** The same paper marks its threshold sweep as its most surprising finding, then restructures the entire second half of the paper around the fact



that removing its own binary search improves results. Most papers would have quietly adopted the better configuration and presented it as the design. Making the reversal the narrative buys more credibility than the original contribution would have, and it is the single most imitable rhetorical move in this window.

**Pre-empt the critique you are personally famous for.** 1.1's benchmark-validity paragraph applies the first author's own prior work on flawed vulnerability-detection benchmarks to the present paper, concedes the agents may be exploiting superficial cues, declines to claim they understand bugs — and then answers with the one argument that actually holds: this benchmark is the deployment task, not a proxy for some other capability. Disarming your own strongest critic works best when your strongest critic is you.

**Report the negative experiment inside the positive result.** EAHC (1.3) could have presented late fusion as an architectural choice. Instead it reports that early fusion was tried and failed, and *why* — the reasoner anchored on the execution outcome. The design is thereby justified by evidence rather than by taste, and the failed variant is more useful to a reader than the successful one.

**Caveat your own best number before a reviewer does.** 1.1's strongest configuration is flagged by the authors themselves as possibly an artefact of three-to-four-times higher token consumption rather than of architecture. Volunteering the confound costs a small amount of headline strength and buys the reader's trust in every other number in the table.

**Name the trade you are making instead of burying it.** CodeMechanic (1.2) states outright that it spends availability to buy security — a fail-stop guard turns potential remote code execution into a crash. Framing mitigation as explicitly temporary and explicitly incomplete makes it a defensible artefact rather than a weak repair, and pre-empts the obvious objection by conceding it as the design.